

PARTICIPANT STUDY GUIDE

Splunk Fundamentals & Data Onboarding

Day 1 — Participant Study Guide

- Module 1 — Introduction to Splunk
- Module 2 — Splunk User Interface
- Module 3 — Data Onboarding Fundamentals
- Module 4 — Introduction to SPL
- Module 5 — Fields and Knowledge Objects
- Case studies, knowledge checks, SPL quick reference

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech, Bengaluru

Running scenario: **NovaMart** (fictional e-commerce retailer)
Applies to Splunk Enterprise 9.x / 10.x & Splunk Cloud Platform
Edition 1.0 · September 2026

Contents

About This Guide	2
The Running Scenario: NovaMart	4
Introduction to Splunk	5
1.1 What is Splunk?	5
1.2 Splunk Use Cases	6
1.3 Splunk Enterprise vs Splunk Cloud Platform	7
1.4 Splunk Platform Architecture	8
1.5 Splunk Licensing Overview	10
Splunk User Interface	13
2.1 Navigating Splunk Web and Installed Apps	13
2.2 The Search & Reporting App	14
2.3 Search Modes: Fast, Smart and Verbose	15
2.4 Search History, Job Inspector and Search Controls	15
2.5 Understanding Events, Fields and Timestamps	16
Data Onboarding Fundamentals	18
3.1 The Splunk Data-Ingestion Workflow	18
3.2 Index, Source, Source Type and Host	19
3.3 Getting Data In: Upload, Monitor and Network	20
3.4 Introduction to the Universal Forwarder	21
3.5 Data Validation and Basic Troubleshooting	23
Introduction to Search Processing Language (SPL)	26
4.1 Search Structure	26
4.2 Terms, Boolean Operators, Wildcards and Exact Phrases	27
4.3 Time-Range Selection	28
4.4 Field-Based Filtering	29
4.5 The Search Pipeline and Commands	29
4.6 Core Commands: fields, table, sort, rename, dedup, head	30
4.7 Transforming vs Non-Transforming Commands	32
Fields and Knowledge Objects	35
5.1 Default, Interesting and Discovered Fields	35
5.2 Field Extraction with the Field Extractor (IFX)	36
5.3 Field Extraction with rex	37
5.4 Calculated Values with eval, if and case	39
5.5 Introduction to Tags, Event Types and Lookups	40
Day 1 Summary	44
SPL Quick Reference	45
Glossary	45
Appendix A: Knowledge Check Answers	47

About This Guide

This study guide is your companion for **Day 1 — Splunk Fundamentals and Data Onboarding**. It explains every concept covered in class, shows it in action on a realistic dataset, and ends each module with a knowledge check. Keep it open during the labs: every lab in the separate *Hands-On Lab Guide* points back to a section here.

Learn

Concepts explained in plain language, with diagrams and comparison tables for quick revision.

See it

Worked SPL examples on the NovaMart dataset, with the results you should see in Splunk.

Apply it

Use cases, case studies and knowledge checks that connect each feature to a real job task.

How the boxes work

Box	What it contains
USE CASE	A real-world situation in which the feature is used: who uses it, and why.
EXAMPLE	A worked example with SPL you can type, and the output you should expect.
TIP	Good practice from production Splunk deployments.
WATCH OUT	A common mistake or misconception, and how to avoid it.
CASE STUDY	A longer scenario at NovaMart that ties several features together.
TRY IT	A pointer to the lab exercise that practises the concept.

Conventions

- SPL (Search Processing Language) appears in dark code blocks. **Commands** after a pipe, **Boolean operators and keywords**, **quoted strings** and **functions** are colour-coded in the same way as the Splunk search bar.
- Raw log lines appear in light grey blocks. Splunk configuration files (`.conf`) and shell commands use a teal or green edge.
- NovaMart, its people, customers and IP addresses are fictional. Attacker addresses use the documentation ranges reserved by RFC 5737 (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24).
- All times are **IST (UTC+05:30)**. Set your Splunk user time zone to *Asia/Kolkata* (Lab 0) so that your screen matches this guide.

Day 1 Agenda

Time	Session	Hands-on
09:30 – 10:30	Module 1 — Introduction to Splunk	Lab 0: environment check
10:30 – 10:45	<i>Break</i>	
10:45 – 11:30	Module 2 — Splunk User Interface	Lab 1: navigate the Splunk interface
11:30 – 13:00	Module 3 — Data Onboarding Fundamentals	Lab 2: upload and index logs Lab 3: identify host, source and source type
13:00 – 13:45	<i>Lunch</i>	

Time	Session	Hands-on
13:45 – 15:15	Module 4 — Introduction to SPL	Lab 4: basic searches Lab 5: filter and format results
15:15 – 15:30	<i>Break</i>	
15:30 – 16:45	Module 5 — Fields and Knowledge Objects	Lab 6: field extraction and calculated fields
16:45 – 17:30	Capstone — NovaMart incident investigation, then recap	Lab 7: case-study investigation

Learning Outcomes

By the end of Day 1 you will be able to:

1. Explain what Splunk is, the problems it solves, and the difference between Splunk Enterprise and Splunk Cloud Platform.
2. Describe the roles of forwarders, indexers, search heads and the deployment server, and how Splunk licensing is measured.
3. Navigate Splunk Web, choose the right search mode, and use search history, the Job Inspector and job controls.
4. Onboard data by uploading files, monitoring files and directories, and receiving network data, with the correct index, source, source type and host.
5. Validate new data and troubleshoot common onboarding problems (timestamps, line breaking, wrong metadata).
6. Write SPL searches that use terms, Boolean logic, wildcards, phrases, time ranges and field filters.
7. Shape results with `fields`, `table`, `sort`, `rename`, `dedup` and `head`, and tell transforming commands from non-transforming ones.
8. Extract new fields with the Field Extractor and `rex`, and create calculated values with `eval`, `case` and `if`.
9. Explain what tags, event types and lookups are, and create a simple one of each.

The Running Scenario: NovaMart

NovaMart is a fictional online retailer based in Bengaluru that sells electronics, fashion, home goods, books, sports equipment and beauty products across India. You have just joined its *Platform Engineering* team. The team has adopted Splunk, and your first job is to bring NovaMart's machine data into it and start answering questions.

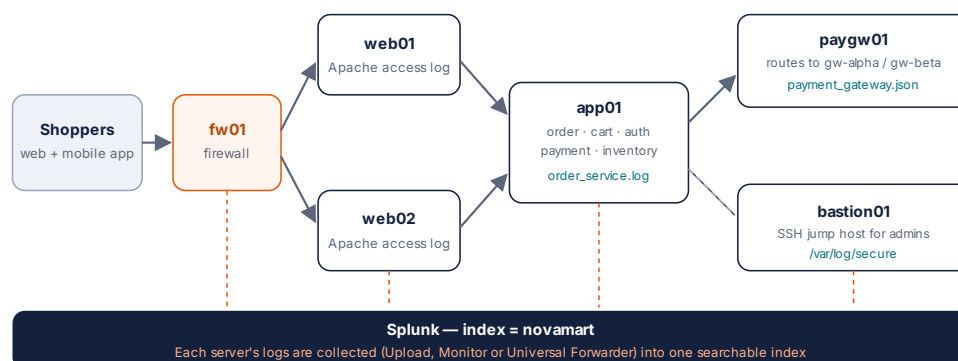


Figure 0.1 — NovaMart platform and the log sources used throughout Day 1

The dataset

The dataset covers **one week, 14–20 September 2026**. On **Friday 18 September at 19:00** NovaMart launched its *Festive Sale*, and evening traffic on the site jumped roughly ninefold. Several things happened during the week, and you will uncover all of them with Splunk.

File	Host	Source type	Events	Contents
web/web01_access.log	web01	access_combined	10,856	Apache combined-format web access logs: page views, cart actions and checkouts, plus bots and scanners
web/web02_access.log	web02	access_combined	11,785	
app/order_service.log	app01	novamart:app	3,042	Application log (ISO timestamps, log level, service, key=value pairs and free text)
payment/payment_gateway.json	paygw01	novamart:payment	806	One JSON object per payment attempt
auth/secure.log	bastion01	linux_secure	1,082	Linux SSH and sudo authentication log
network/firewall.log	fw01	novamart:firewall	300	Firewall syslog, sent over UDP in the optional network-input lab
lookups/*.csv , config/*.conf	—			Product catalogue, HTTP status and threat-intel lookups; reference <code>inputs / outputs / props / transforms.conf</code>

Introduction to Splunk

What Splunk is · Use cases · Enterprise vs Cloud · Architecture · Licensing

AFTER THIS MODULE YOU CAN

- explain machine data and why organisations centralise it in Splunk
- map IT Operations, Security, Observability and Business Analytics use cases to data sources
- compare Splunk Enterprise with Splunk Cloud Platform
- describe what the forwarder, indexer, search head and deployment server each do
- explain how Splunk licensing is measured and what happens when you exceed it

1.1 What is Splunk?

Splunk is a data platform for collecting, indexing, searching, analysing and visualising **machine data**: the records that servers, applications, network devices, cloud services, sensors and security tools generate as they run. Splunk was founded in 2003; its name comes from *spelunking* (exploring caves), an image of digging through huge volumes of data to find what matters. Cisco completed its acquisition of Splunk in March 2024.

The core idea is simple: **collect everything once, then ask any question later**. Splunk stores data in its raw form and applies structure *when you search*. This approach is called **schema-on-read**, and it is what makes Splunk different from a traditional database.

Traditional database (schema-on-write)

- You design tables and columns *before* loading data
- Data that does not fit the schema is rejected or transformed
- A new question often needs a schema change and a reload

Splunk (schema-on-read)

- Data is indexed as it arrives, in its raw form
- Fields are extracted at search time, so structure can change whenever you need it to
- A new question needs only a new search

What is machine data?

Machine data is usually **time-stamped**, **high-volume**, **semi-structured or unstructured**, and comes from **many different formats**. Here are three NovaMart events that describe the same customer journey in three formats:

WEB SERVER — APACHE COMBINED FORMAT (WEB02)

```
117.26.150.82 - - [18/Sep/2026:20:07:38 +0530] "POST /checkout?order_id=ORD-550497&JSESSIONID=WA56QA1WDQ2J HTTP/1.1" 503 672 "https://www.novamart.in/cart.do?action=view" "Mozilla/5.0 (Linux; Android 14; SM-A546E) ..."
```

APPLICATION LOG — TIMESTAMP, LEVEL, SERVICE, KEY=VALUE (APP01)

```
2026-09-18T20:08:08.464+05:30 ERROR [payment-service] txn_id=TXN-101536 Payment gateway gw-beta timed out after 30000 ms for order ORD-550497 attempt 3 of 3
```

PAYMENT GATEWAY — JSON (PAYGW01)

```
{ "timestamp": "2026-09-18T20:08:08.461+05:30", "gateway": "gw-beta", "order_id": "ORD-550497", "amount": 18999.0, "payment_method": "CARD", "result": "timeout", "latency_ms": 30464, ... }
```

Each format is different, but all three carry a **timestamp** and a shared value (`ORD-550497`). Splunk can search across all three at once and connect them. Try `index=novamart ORD-550497` once the data is loaded: it returns four events from three sources. That ability to link events from different systems is where most of Splunk's value comes from.

The Splunk workflow

Stage	What happens	NovaMart example
Collect	Data arrives from files, network ports, APIs, scripts or forwarders	Apache logs from web01 and web02
Index	Data is broken into events, time-stamped, compressed and stored with an index for fast retrieval	22,641 web events stored in index <code>novamart</code>
Search	SPL retrieves and filters events, and fields are extracted on the fly	<code>status=503</code> returns 35 failed checkouts
Analyse	Statistics, correlation and trends	Failed revenue by payment method
Visualise / Act	Reports, dashboards, alerts and automated responses	Alert when checkout 5xx errors exceed 5 per minute

1.2 Splunk Use Cases

Organisations use Splunk in four broad areas. The same data often serves several of them: NovaMart's web access log matters to operations engineers, security analysts, SREs and the business team.

IT Operations

Find and fix problems in servers, applications and networks faster, and reduce the **mean time to resolution (MTTR)**.

- Troubleshoot application errors and outages
- Track capacity and performance
- Analyse the impact of changes and deployments
- Premium: *Splunk IT Service Intelligence (ITSI)*

Security

Detect, investigate and respond to threats. Splunk is widely used as a **SIEM** (security information and event management) platform.

- Brute-force and credential-stuffing detection
- Threat hunting and incident investigation
- Compliance reporting (PCI DSS, ISO 27001, RBI guidelines)
- Premium: *Splunk Enterprise Security, Splunk SOAR*

Observability

Understand the internal state of modern distributed systems from their **logs, metrics and traces**.

- Service health and SLO/error-budget tracking
- Latency and error-rate monitoring (RED metrics)
- Correlating logs with infrastructure metrics and APM traces
- Premium: *Splunk Observability Cloud, Splunk APM*

Business Analytics

Turn operational data into business insight in near real time, without waiting for a data warehouse load.

- Orders, revenue and conversion funnels
- Customer behaviour and product popularity
- Payment success rates by gateway or method
- Real-time business KPIs on dashboards

USE CASE · IT OPERATIONS

At 20:10 on sale night, the NovaMart on-call engineer gets reports that checkout is failing. Instead of logging in to six servers and running `grep`, she runs one Splunk search across all web servers and sees that every failure is an HTTP 503 on `/checkout`, and that failures started at 20:07.

```
index=novamart sourcetype=access_combined status=503 | stats count by host, uri_path
```

USE CASE · SECURITY

The security team wants to know whether anyone is guessing passwords on the SSH bastion host. One search shows the addresses behind the most failed logins:

```
index=novamart sourcetype=linux_secure "Failed password" | rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | top src_ip
```

Result: `203.0.113.77` is responsible for 414 of the 466 failed passwords (88.8%).

USE CASE · OBSERVABILITY

The SRE team tracks payment latency per gateway as a service-level indicator. A sudden jump in average latency on `gw-beta` is an early warning, even before customers complain.

```
index=novamart sourcetype=novamart:payment | stats avg(latency_ms) AS avg_latency max(latency_ms) AS max_latency by gateway
```

USE CASE · BUSINESS ANALYTICS

The category manager asks: "How much revenue did each region generate this week?" The answer comes straight from the application log, with no ETL job:

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS | stats sum(amount) AS revenue by region | sort - revenue
```

North (₹17.7 lakh) and South (₹17.2 lakh) lead, followed by West and East.

Use case	Typical data sources	Typical questions
IT Operations	Web/app logs, OS logs, performance counters	What broke? When? Which hosts are affected? What changed?
Security	Authentication logs, firewall, proxy, EDR, cloud audit logs	Who is attacking us? Did they get in? What did they do?
Observability	Metrics, traces, structured application logs	Are we meeting our SLOs? Where is latency coming from?
Business Analytics	Order/transaction logs, clickstream, payment events	How much did we sell? Where do customers drop out of the funnel?

1.3 Splunk Enterprise vs Splunk Cloud Platform

Splunk's core platform comes in two deployment models. The search experience and SPL are the same in both. What differs is **who runs the infrastructure** and **how much low-level access you have**.

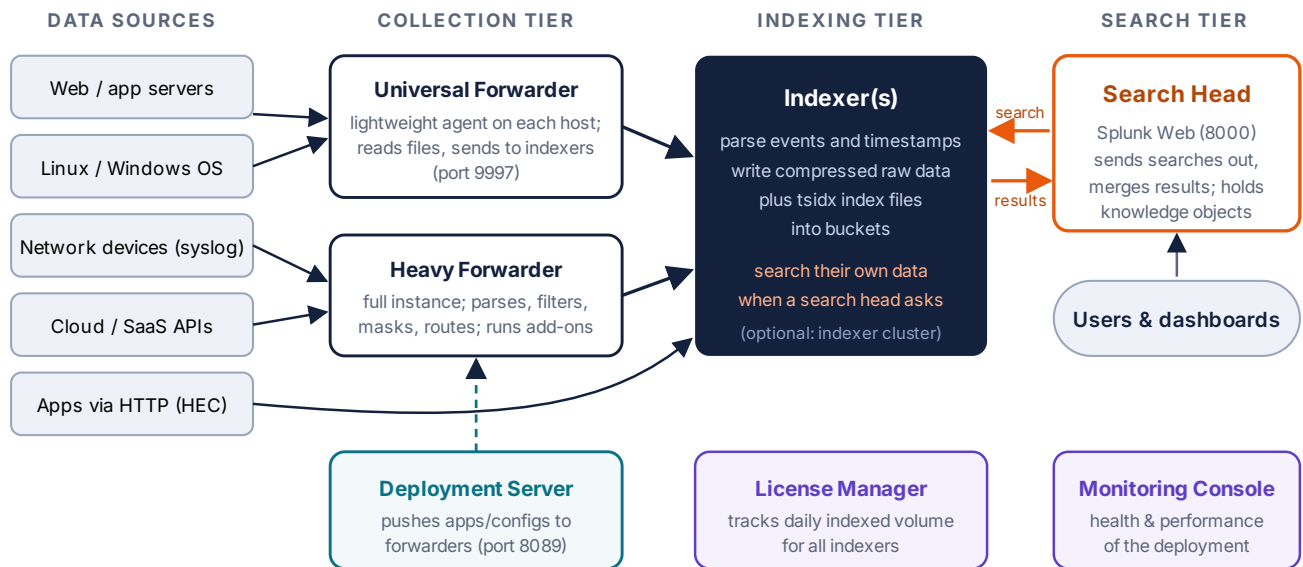
Aspect	Splunk Enterprise	Splunk Cloud Platform
Deployment	Self-managed software you install on your own servers, on-premises or in your own public-cloud account	Software as a service (SaaS), hosted and operated by Splunk
Who manages the infrastructure	You: hardware, OS, upgrades, scaling, backups, clustering	Splunk: patching, upgrades, availability and scaling are handled for you
Access to configuration	Full: <code>.conf</code> files, CLI, file system	No direct file-system or CLI access to the cloud stack; configure through Splunk Web and the Admin Config Service (ACS)
Apps and add-ons	Install any app	Apps must pass Splunk Cloud vetting before installation
Getting data in	Forwarders, network inputs, HEC, uploads	Forwarders (with a Splunk Cloud credentials package), HEC, uploads; forwarders still run in your environment
Time to value	Longer: you must plan, size and build the deployment	Faster: the stack is provisioned for you
Best fit	Strict data-residency or air-gapped needs, deep customisation, existing infrastructure teams	Organisations that want to focus on using the data rather than running the platform

TIP

For learning and labs, you can use either a **Splunk Enterprise trial** (free for 60 days and up to 500 MB/day of indexing, installed on your laptop or in Docker) or a **Splunk Cloud Platform trial**. Everything you learn today works in both. The only exception is *monitoring local files*, which needs a Splunk instance, or a forwarder, that can see the files.

1.4 Splunk Platform Architecture

Splunk is built from **components**, each of which is a Splunk instance configured for a particular role. In a small deployment one server can do everything. In large deployments the roles are split across many servers so that each can scale on its own.



Management components (bottom row) are often combined on one server in small and medium deployments.

Figure 1.1 — Splunk components and how data and searches flow between them

Component	Role	Key facts
Forwarder	Collects data at the source and sends it to indexers. The Universal Forwarder (UF) is a small, separate installer with no web interface. The Heavy Forwarder (HF) is a full Splunk Enterprise instance that can parse, filter, mask and route data before sending it on.	Sends to indexers on TCP 9997 by convention. Uses little CPU and memory. Buffers data if the indexer is unavailable.
Indexer	Receives data, breaks it into events, extracts timestamps, and writes compressed raw data and index files (<i>tsidx</i>) to disk in buckets . It also runs its share of every search.	Buckets age from <i>hot</i> → <i>warm</i> → <i>cold</i> → <i>frozen</i> . Frozen data is deleted or archived. Indexers can be clustered to replicate data.
Search Head	Hosts Splunk Web, where users write searches. It sends each search to the indexers (<i>distributed search</i>), merges the results, and stores knowledge objects such as saved searches, dashboards, field extractions and lookups.	Web UI on port 8000 . Can be clustered for high availability.
Deployment Server	Centrally manages the configuration of many forwarders (and other clients). You group clients into <i>server classes</i> and assign <i>deployment apps</i> to each group.	Clients phone home on the management port 8089 . The UI is <i>Forwarder Management</i> (called <i>Agent Management</i> in newer releases).

Deployment topologies

Single instance

One server does indexing and searching. Suits labs, proofs of concept and small teams at under about 20 GB/day.

Today's labs use this model.

Distributed

Dedicated search heads and indexers, with universal forwarders on data sources. Suits departmental and enterprise deployments.

Clustered

An indexer cluster (data replication) plus a search head cluster (high availability). Suits mission-critical, large-scale deployments.

Common ports

Port	Used for	Notes
8000	Splunk Web (browser UI)	HTTPS can be enabled; often changed in production
8089	Management port (REST API, deployment server, distributed search)	Used by every Splunk instance, including forwarders
9997	Receiving data from forwarders (splunktcp)	Must be enabled on the indexer: <i>Settings</i> → <i>Forwarding and receiving</i>
8088	HTTP Event Collector (HEC)	Token-based HTTP/HTTPS ingestion from applications
514 (UDP/TCP)	Syslog by convention	Ports below 1024 need root; labs use 5514 instead
8191	KV Store	Internal database used by apps and lookups

EXAMPLE · TRACING ONE EVENT THROUGH THE ARCHITECTURE

1. A shopper checks out, and Apache on **web02** writes a line to `/var/log/httpd/access_log`.
2. The **Universal Forwarder** on web02 notices the new line and sends it, tagged with `host=web02`, `sourcetype=access_combined` and `index=novamart`, to the **indexer** on port 9997.
3. The indexer recognises the timestamp `[18/Sep/2026:20:12:05 +0530]`, compresses the event and writes it to a hot bucket of index `novamart`.
4. An engineer searches `status=503` on the **search head**. The search head sends the search to all indexers, each one scans its buckets, and the search head merges and displays the matching events.

1.5 Splunk Licensing Overview

Splunk licensing controls **how much you can use the platform**. The two main models are:

Ingest-based pricing

Measured in **GB of data indexed per day**. Every byte of raw data written to an index counts toward the daily quota. The count resets at **midnight**, by the License Manager's clock.

Workload-based pricing

Measured in **compute capacity**: virtual CPUs for Splunk Enterprise, or Splunk Virtual Compute (SVC) units for Splunk Cloud Platform. You can ingest more data, and what you pay for is the processing power used to search it.

What counts toward ingest licensing?

Counts	Does not count
Size of raw data as it is indexed (before compression)	Splunk's own internal indexes: <code>_internal</code> , <code>_audit</code> , <code>_introspection</code> and similar
Data from every input: files, network, HEC, scripts, forwarders	Metadata Splunk adds (host, source, source type), and disk-space overhead
Data indexed and then deleted: the licence is used at index time	Summary-index data written with the <code>stash</code> source type

License types

Type	Description
Enterprise	Purchased licence with a daily volume (or workload) entitlement; all features enabled.
Enterprise Trial	Installed automatically with Splunk Enterprise. Valid for 60 days at up to 500 MB/day , with all features.
Free	Up to 500 MB/day with no expiry, but with no user accounts or roles, no alerting or scheduled searches, and no distributed search or clustering. Suitable only for personal learning.
Developer	For building and testing apps outside production, with a higher daily limit for a fixed term.
Forwarder	Built into the Universal Forwarder; lets forwarders send data without using indexing licence themselves.

Warnings and violations

If the data indexed in a day exceeds the licensed volume, the License Manager records a **license warning**. Too many warnings within a rolling window turn into a **license violation**. The exact threshold, and whether search is restricted, depend on the licence type, the licence size and the Splunk version.

WATCH OUT

Splunk never stops indexing because of a licence overage, so no data is lost. Where enforcement applies, it is *searching* (of non-internal indexes) that may be blocked until the violation clears. Watch your usage in *Settings* → *Licensing* or in the Monitoring Console's licence-usage report.

EXAMPLE · ESTIMATING NOVAMART'S DAILY LICENCE NEED

The lab dataset covers 7 days and is about 6.5 MB in total, roughly 0.9 MB/day. That is tiny. Production NovaMart is much larger:

Source	Instances	Avg per instance per day	Daily total
Web access logs	24 web servers	1.8 GB	43.2 GB
Application logs	12 app servers	2.5 GB	30.0 GB
Payment JSON	2 gateways	1.2 GB	2.4 GB
Linux secure logs	60 hosts	40 MB	2.4 GB
Firewall syslog	4 firewalls	3.0 GB	12.0 GB
Total (plus about 20% headroom for sale-day peaks)			90 GB → ~110 GB/day

Sizing rule of thumb: plan for peak days (sales, incidents, attacks), not averages. A brute-force attack can multiply authentication-log volume by 100.

CASE STUDY 1

One platform, four teams: NovaMart's first week with Splunk

Situation. Before Splunk, each NovaMart team had its own tool. The web team used `grep` on servers, the security team read the bastion logs by hand once a week, the SRE team watched a basic uptime checker, and the business team waited for a nightly data-warehouse report.

What changed. The platform team sent all five log sources into a single index, `novamart`. Within the first week:

- **IT Ops** traced the Festive Sale checkout outage (Friday 20:07–20:45) to one payment gateway in minutes rather than hours.
- **Security** found a brute-force attack on Thursday at 02:11 that ended in a *successful* login, followed by the creation of a new user account. Nobody had noticed it.
- **SRE** found that `gw-beta` latency is normally around 400 ms but reached 30 seconds during the outage, and started tracking it as a service-level indicator.
- **Business** saw live Festive Sale revenue by category and region on the evening of the sale, not the next morning.

Lesson. Splunk pays off most when data from several systems and teams sits in one place. The same event, such as a failed checkout, answers an operations question, an SRE question and a business question. You will investigate both incidents yourself in **Lab 7**.

KNOWLEDGE CHECK · MODULE 1

1. What does "schema-on-read" mean, and why does it matter for machine data?
2. Which component would you install on 200 Linux servers to collect their logs? Which component would you use to manage those 200 installations centrally?
3. A team needs to mask credit-card numbers *before* data leaves the data centre. Should they use a Universal Forwarder or a Heavy Forwarder? Why?
4. True or false: when a Splunk Enterprise licence is exceeded, Splunk stops indexing new data.
5. Name two differences between Splunk Enterprise and Splunk Cloud Platform that affect a Splunk administrator.

Answers are in Appendix A.

Splunk User Interface

Splunk Web · Apps · Search & Reporting · Search modes · Job Inspector · Events, fields and timestamps

AFTER THIS MODULE YOU CAN

- find your way around Splunk Web and switch between installed apps
- identify every part of the Search & Reporting app
- choose between Fast, Smart and Verbose search modes
- re-run past searches, control running jobs and read the Job Inspector
- explain what events, fields and timestamps are, and how Splunk displays time

2.1 Navigating Splunk Web and Installed Apps

Splunk Web is the browser interface to Splunk. It runs on the search head, by default at `http://<server>:8000`. After logging in you land on the **Home** page, or on your role's default app.

Area	What you find there
Splunk bar (top, dark)	<i>App</i> menu (switch apps), <i>Administrator</i> /user menu (Account settings, Preferences such as time zone and default app, Logout), <i>Messages</i> (system notices, licence warnings), <i>Settings</i> (all configuration), <i>Activity</i> (Jobs, Triggered Alerts), <i>Help</i> , and <i>Find</i> (search across knowledge objects)
App bar (below the Splunk bar)	The navigation menu of the current app, e.g. <i>Search</i> , <i>Analytics</i> , <i>Datasets</i> , <i>Reports</i> , <i>Alerts</i> , <i>Dashboards</i> in Search & Reporting
Apps panel (Home page)	Installed apps you are allowed to see; <i>Find more apps</i> opens Splunkbase
Settings menu	Knowledge (searches, fields, lookups, event types, tags), Data (inputs, indexes, forwarding and receiving, source types), System (licensing, server settings), Users and authentication

Apps and add-ons

App

A packaged collection of **user-facing** functionality: dashboards, reports, navigation menus and knowledge objects. Examples: *Search & Reporting*, *Splunk Enterprise Security*, *Splunk App for Lookup File Editing*.

Add-on (TA, "technology add-on")

Usually has **no UI**. It provides inputs, source-type definitions and field extractions for one technology. Examples: *Splunk Add-on for Unix and Linux*, *Splunk Add-on for Microsoft Windows*, *Splunk Add-on for AWS*.

Apps and add-ons are downloaded from **Splunkbase** (splunkbase.splunk.com), where thousands are published. Everything you create, such as a saved search or a field extraction, lives **inside an app context**. That is why a report you saved in one app may not show up in another unless its permissions are shared.

TIP

Set your **default app** to *Search & Reporting* and your **time zone** to *Asia/Kolkata* under *User menu* → *Preferences*. Most people in this course will spend their whole day in this one app.

2.2 The Search & Reporting App

Search & Reporting (often just called "the Search app") is where you write SPL, explore events and build reports. Its main page has the following parts:

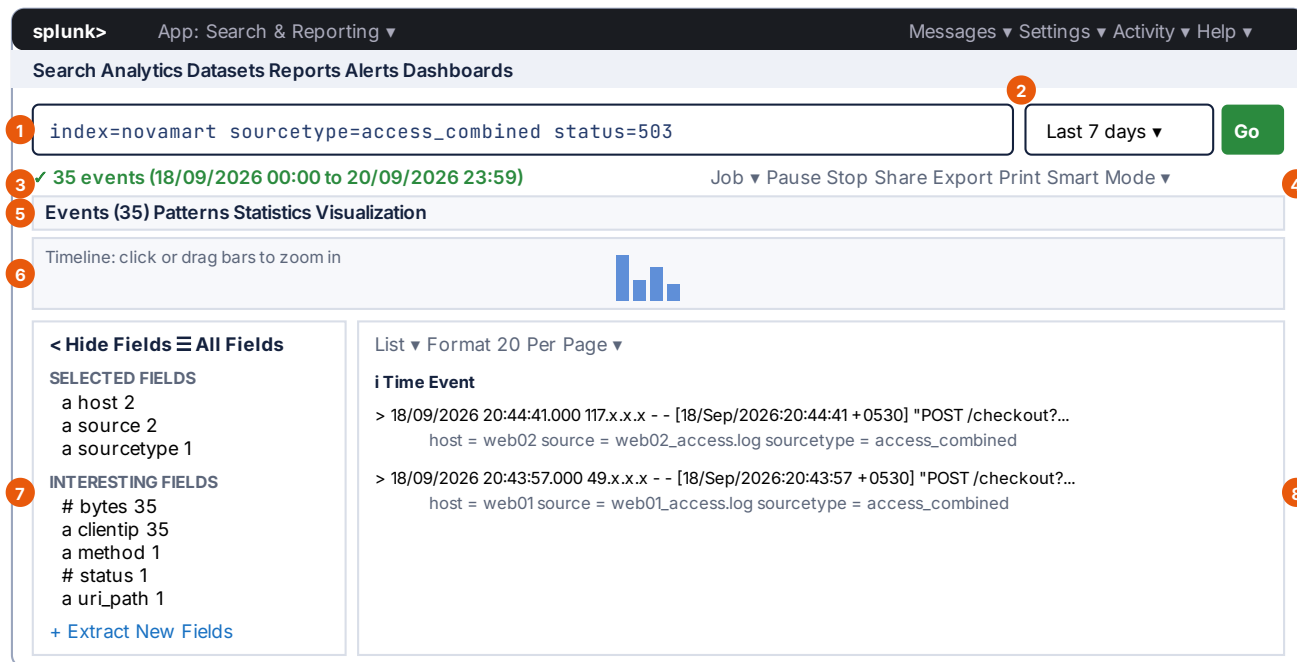


Figure 2.1 — Anatomy of the Search page (simplified)

#	Element	Purpose
1	Search bar	Type SPL here. The Search Assistant suggests commands, shows syntax and lists matching past searches as you type.
2	Time range picker	Limits the search to a time window: presets, relative, real-time, date and time range, or advanced. The most important efficiency control you have.
3	Search status	Number of events matched and the time range actually searched.
4	Job controls and mode	Job menu, pause, stop, share, export, print, and the search mode selector.
5	Results tabs	Events (raw events), Patterns (automatically grouped similar events), Statistics (tables from transforming commands), Visualization (charts).
6	Timeline	Histogram of matching events over time. Click a bar to focus on that period, or drag to select a range, without re-running the search.
7	Fields sidebar	Selected, interesting and all fields. Click a field to see its top values and quick reports.
8	Events viewer	List, Raw or Table view. Expand an event with > to see its fields and <i>Event Actions</i> (Build Event Type, Extract Fields, Show Source).

EXAMPLE · DRILLING DOWN WITHOUT WRITING SPL

Search `index=novamart sourcetype=access_combined` over *All time*. In the fields sidebar, click **status**. The pop-up shows the top values: `200` (98.3%), `404`, `402`, `500`, `503`... Click **503**. Splunk adds `status=503` to your search and re-runs it. Now click the tallest bar in the timeline, and the view zooms to 18 September. You have found the outage window in three clicks.

2.3 Search Modes: Fast, Smart and Verbose

The search mode controls **how much work Splunk does** to discover fields and keep event data. It changes speed, but it does not change which events match.

Mode	Field discovery	Event data for transforming searches	Use it when
Fast	Off. Only default fields and fields named in the search are returned.	Not returned (Statistics only)	You know exactly what you want, e.g. a report or dashboard search. Fastest.
Smart (default)	On for non-transforming searches; off for transforming searches.	Not returned	Everyday use. Splunk picks the sensible behaviour for you.
Verbose	On, always, for all fields.	Returned: the Events tab is populated even for <code>stats</code> or <code>top</code>	Exploring unfamiliar data or debugging a search. Slowest.

WATCH OUT

"My fields sidebar is empty!" In Fast mode, Splunk does not discover fields such as `clientip` or `uri_path`, so they will not appear in the sidebar. They still work if you use them in the search itself. Switch to Smart or Verbose mode to see them.

EXAMPLE · SAME SEARCH, THREE MODES

Run `index=novamart sourcetype=access_combined status=404 | top clientip` in each mode and open the **Job Inspector** each time. In Fast and Smart mode, only the Statistics tab has results. In Verbose mode, the Events tab also shows all 240 raw 404 events, and the run takes noticeably longer. `192.0.2.66` is the top result, with 153 of the 240 errors.

2.4 Search History, Job Inspector and Search Controls

Search history

The Search app home page has a **Search History** panel that lists your recent searches, with a filter box and a time filter. Click any past search to re-run it with the time range it originally used. The Search Assistant also offers *matching searches* from your history as you type.

Search jobs

Every search you run creates a **search job**: a process on the search head, plus result artefacts on disk that are identified by a *search ID (SID)*. By default a completed ad-hoc job is kept for **10 minutes**, which is its *time to live (TTL)*. After that its results expire and the search must run again.

Control	What it does
Pause / Resume	Temporarily stops a long-running search without losing the progress made so far.
Stop	Ends the search and keeps the results found so far.
Finalize	Stops collecting events and runs the rest of the pipeline on what has been collected so far.
Send to Background	Lets the search keep running while you do something else. You are notified when it finishes, and it appears under <i>Activity</i> → <i>Jobs</i> .

Control	What it does
Share	Extends the job's lifetime to 7 days , gives other users read permission, and gives you a link to the results.
Export	Downloads results as CSV, JSON, XML or raw events.
Print	Prints or saves as PDF.
Activity → Jobs	Lists all your jobs, where you can inspect, extend their lifetime, pause, stop or delete them. Administrators see everyone's jobs.

The Job Inspector

Open **Job** → **Inspect Job** to see how your search actually ran. It is the main tool for understanding and speeding up slow searches.

JOB INSPECTOR HEADER (EXAMPLE)

This search has completed and has returned 35 results by scanning 35 events in 0.412 seconds

Item	Meaning
Execution costs	Time spent in each search component, e.g. <code>command.search.index</code> (looking up the index files), <code>command.search.rawdata</code> (reading raw data), <code>command.search.kv</code> (field extraction), and each command in your pipeline.
<code>scanCount</code>	Number of events Splunk had to read from disk.
<code>eventCount</code>	Number of events that matched the search.
<code>resultCount</code>	Number of results returned at the end of the pipeline.
<code>runDuration</code>	Total run time in seconds.
<code>normalizedSearch</code>	The search as Splunk actually ran it, after expanding macros, event types and tags. Very useful for debugging.

TIP · READING EFFICIENCY FROM THE JOB INSPECTOR

Compare `scanCount` with `eventCount`. When they are close, your search terms matched the index files well (efficient). When `scanCount` is far higher, Splunk read many events only to throw them away. Add `index=`, `sourcetype=`, more specific terms, or a narrower time range.

2.5 Understanding Events, Fields and Timestamps

Event

A single record of activity, usually one line of a log file (sometimes several lines, e.g. a Java stack trace). Every event has a timestamp.

Field

A **name=value** pair that describes part of an event, such as `status=503` or `host=web02`. Most fields are extracted at **search time**.

Timestamp (`_time`)

When the event happened. Splunk extracts it from the event at index time and stores it as **UNIX epoch time** (seconds since 1 Jan 1970 UTC).

Anatomy of an event

Here is one NovaMart web event, with the fields Splunk extracts automatically for the `access_combined` source type underlined in colour:

```
117.26.150.82 - - [18/Sep/2026:20:07:38 +0530] "POST /checkout?order_id=ORD-550497&JSESSIONID=WA56QA1WDQ2J
HTTP/1.1" 503 672 "https://www.novamart.in/cart.do?action=view" "Mozilla/5.0 (Linux; Android 14; SM-A546E)
..."
```

clientip _time method uri_path uri_query (plus order_id, JSESSIONID) status

bytes referer useragent

On top of these, every event carries **default fields** that Splunk adds itself: `host=web01`, `source=web01_access.log`, `sourcetype=access_combined`, `index=novamart`, `_raw` (the full original text), `_time`, `linecount`, `punct`, `splunk_server` and others. Module 5 covers fields in detail.

How Splunk handles time

- At **index time**, Splunk finds the timestamp in the raw text (or uses rules you configure), converts it to UTC epoch, and stores it in `_time`.
- At **search time**, `_time` is displayed in **your user time zone** (Preferences). Two analysts in Bengaluru and London see the same event at different clock times, and both are correct.
- If the raw timestamp has **no time zone** (as in syslog: `Sep 17 02:34:51`), Splunk assumes the time zone of the indexer or forwarder, unless you set `TZ` in `props.conf`. This is the most common onboarding mistake; see Module 3.
- If no timestamp can be found, Splunk falls back to the timestamp of the previous event from the same source, and ultimately to the time the event was indexed (`_indextime`).

Raw timestamp in the event	Has a time zone?	Stored as <code>_time</code> (UTC)
<code>[18/Sep/2026:20:07:38 +0530]</code>	Yes (+05:30)	2026-09-18 14:37:38 UTC
<code>2026-09-18T20:08:08.464+05:30</code>	Yes (+05:30)	2026-09-18 14:38:08.464 UTC
<code>Sep 17 02:34:51</code>	No (and no year!)	Depends on the <code>TZ</code> setting; the year is inferred

TRY IT

Lab 1 takes you through Splunk Web, the Search app, search modes, search history and the Job Inspector, step by step.

KNOWLEDGE CHECK · MODULE 2

- Which search mode would you pick to explore a data source you have never seen before? Why?
- You ran a search 20 minutes ago and now its results have "expired". Why? How could you have kept them?
- In the Job Inspector, `scanCount` = 2,000,000 and `eventCount` = 35. What does that tell you, and what would you do?
- What is the difference between an *app* and an *add-on*?
- Your colleague in London and you in Bengaluru open the same event, and the displayed times differ by 4 h 30 m (5 h 30 m in winter). Is something broken?

Data Onboarding Fundamentals

Ingestion workflow · Index, source, source type, host · Upload, Monitor, Network · Universal Forwarder · Validation and troubleshooting

AFTER THIS MODULE YOU CAN

- describe the input, parsing and indexing phases of the Splunk data pipeline
- choose the right index, source, source type and host for a new data source
- onboard data by uploading files, monitoring files and directories, and receiving network data
- explain what the Universal Forwarder does and how it is configured
- validate newly onboarded data and troubleshoot the most common problems

3.1 The Splunk Data-Ingestion Workflow

Every piece of data travels through the same **data pipeline** before it can be searched. Knowing the phases tells you *where* each setting applies and *which* component does the work.

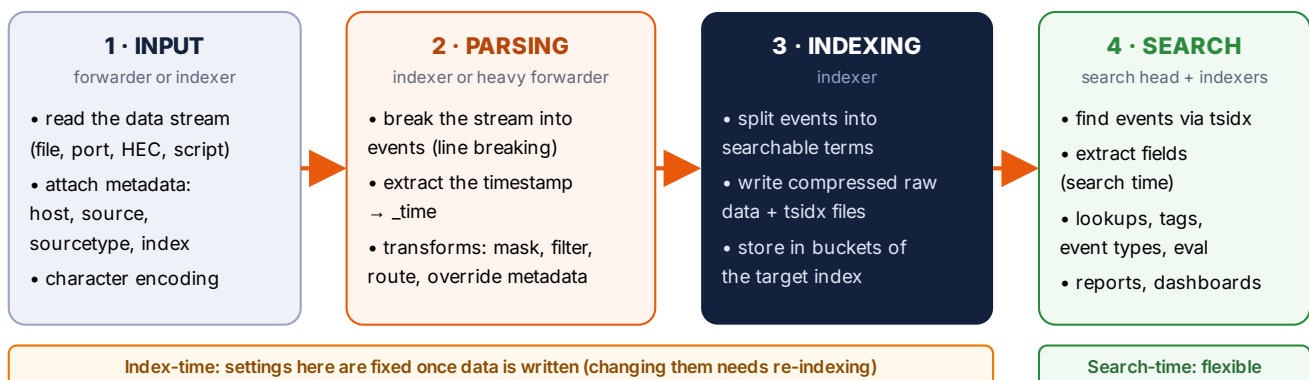


Figure 3.1 — The Splunk data pipeline

WATCH OUT · INDEX TIME VS SEARCH TIME

Whatever happens during input, parsing and indexing is **permanent** for data already indexed: event boundaries, `_time`, host, source, source type, index. If you get the timestamp or line breaking wrong, you must fix the configuration, delete the data and **re-index** it. Search-time settings (field extractions, lookups, tags) can be changed at any time. That is why you should always test a new source in a **test index** first.

Buckets and index storage

Bucket	State	Typical storage
Hot	Being written to now; searchable	Fastest disk (SSD)
Warm	Recently rolled from hot; read-only; searchable	Fast disk
Cold	Older data; read-only; searchable	Cheaper, slower disk

Bucket	State	Typical storage
Frozen	Past its retention period: deleted by default, or archived if configured; <i>not</i> searchable	Archive storage or deleted
Thawed	Archived data restored so it can be searched again	Any

3.2 Index, Source, Source Type and Host

Four metadata fields are attached to every event during the input phase. They are the most useful search filters you have, so set them deliberately.

Field	Meaning	NovaMart examples	How it is set
<code>index</code>	The data store the event is written to. It controls retention , access control (roles are granted per index) and search performance.	<code>novamart</code> , <code>main</code> , <code>_internal</code>	Input setting (<code>index =</code>); default is <code>main</code>
<code>source</code>	Where the event came from: the file path, network port, or script name.	<code>/opt/novamart/app/order_service.log</code> , <code>udp:5514</code>	Automatic, from the input
<code>sourcetype</code>	The format of the data. It tells Splunk how to break events, find timestamps and extract fields.	<code>access_combined</code> , <code>linux_secure</code> , <code>novamart:app</code>	Chosen or automatically detected at input; defined in <code>props.conf</code>
<code>host</code>	The device that generated the event.	<code>web01</code> , <code>app01</code> , <code>bastion01</code>	Defaults to the collecting machine's host name; can be a constant, a regex on the path, or a path segment

Index good practice

- Do **not** send everything to `main`. Create indexes by retention need and by audience (e.g. `web`, `security`, `payments`).
- Index names are lowercase letters, digits, `_` and `-`, and must not start with `_` or `-` (those are reserved for internal indexes).
- Always create a **test index** for trying out new sources.

Source type good practice

- Use Splunk's **pretrained** source types where the format matches, e.g. `access_combined`, `syslog`, `linux_secure`, `_json`, `csv`.
- Name custom ones `vendor:product:type`, e.g. `novamart:app` or `cisco:asa`.
- One source type per **format**, not per server.

EXAMPLE · WHY METADATA MAKES SEARCHES FAST

The second search tells Splunk exactly where to look and what to match, so it scans far fewer events and returns exactly the 35 HTTP 503 errors:

```
503
```

```
index=novamart sourcetype=access_combined status=503
```

The first searches every index your role can access for the term 503. It would also match events where 503 appears in a byte count, an order ID or a port number.

Internal indexes you should know

Index	Contains
<code>main</code>	Default index for data with no index specified
<code>_internal</code>	Splunk's own logs (<code>splunkd.log</code> , <code>metrics.log</code> , <code>license_usage.log</code>). Your first stop for troubleshooting.
<code>_audit</code>	Audit trail: logins, searches run, configuration changes
<code>_introspection</code>	Resource usage data (CPU, memory, disk) of Splunk processes
<code>summary</code>	Default destination for summary indexing

3.3 Getting Data In: Upload, Monitor and Network

In Splunk Web, all inputs start from **Settings** → **Add Data** (or *Add Data* on the Home page). There are three options: **Upload**, **Monitor** and **Forward**.

Method	How it works	Best for	Limits / notes
Upload	You pick a file in your browser; Splunk indexes it once .	Testing a source type; one-off analysis of a log someone emailed you; training	Up to 500 MB per file through Splunk Web. Not continuous.
Monitor	Splunk watches a file or directory and continuously indexes new data as it is written (like <code>tail -f</code>).	Log files on the Splunk server itself; the basis of forwarder inputs	The file must be readable by the Splunk process. Handles log rotation.
Network (TCP/UDP)	Splunk listens on a port and indexes whatever arrives.	Syslog from firewalls, switches and appliances	UDP can lose data. In production, prefer a syslog server or Splunk Connect for Syslog (SC4S) in front of Splunk.
HTTP Event Collector	Applications POST events over HTTP(S) with a token.	Cloud-native apps, serverless, custom code, streaming pipelines	Port 8088 by default. Needs a token.
Forward	A forwarder on another machine collects the data and sends it to Splunk.	Almost all production data collection	Configured on the forwarder or centrally through the deployment server.

The Add Data wizard: step by step

- 1 Select Source.** Choose the file, directory or port. For a monitor input you can also set *Continuously Monitor* or *Index Once*, and allow-list or deny-list file names within a directory.
- 2 Set Source Type.** Splunk shows a **preview** of how it will break events and read timestamps. Pick an existing source type, or adjust *Event Breaks*, *Timestamp* and *Advanced* settings (e.g. `TZ` , `TIME_FORMAT`) and **Save As** a new source type. *This is the most important screen; check the preview carefully.*
- 3 Input Settings.** Choose the **app context**, the **host** (constant value, regular expression on the path, or segment in the path) and the **index**. You can create a new index here.
- 4 Review** your choices, then **Submit**. Splunk offers *Start Searching*, which opens a search for exactly the data you just added.

EXAMPLE · ONBOARDING NOVAMART'S APPLICATION LOG AS A CUSTOM SOURCE TYPE

The file `order_service.log` does not match any pretrained source type. In the *Set Source Type* step the preview shows correct event breaks (one line per event) and the ISO timestamp is detected automatically. To be explicit and repeatable, we set:

```
PROPS.CONF — GENERATED BY "SAVE AS: NOVAMART:APP"

[novamart:app]
SHOULD_LINEMERGE = false
LINE_BREAKER = ([\r\n]+)
TIME_PREFIX = ^
TIME_FORMAT = %Y-%m-%dT%H:%M:%S.%3N%:z
MAX_TIMESTAMP_LOOKAHEAD = 30
KV_MODE = auto
```

`%:z` matches a time-zone offset that contains a colon, such as `+05:30`. `KV_MODE = auto` tells Splunk to extract every `key=value` pair automatically at search time, so `amount`, `status`, `region` and the rest become fields with no further work.

How monitoring works under the hood

- Splunk records a **CRC checksum of the first 256 bytes** of each file, plus a **seek pointer** (how far it has read), in an internal database called the *fishbucket*.
- When a log rotates (`app.log` → `app.log.1`), Splunk recognises the CRC and does not re-index the renamed file.
- Files that start with an identical header (e.g. CSV files with the same column line) can confuse the CRC check. `crcSalt = <SOURCE>` adds the file path to the checksum to fix this.
- Use `whitelist` / `blacklist` (regular expressions) to include or exclude files in a monitored directory, e.g. skip `*.gz` archives.

Receiving network data

```
INPUTS.CONF — UDP INPUT FOR NOVAMART FIREWALL SYSLOG

[udp://5514]
index = novamart
sourcetype = novamart:firewall
connection_host = ip          # host field = sender's IP address (ip | dns | none)
```

TIP

Ports below 1024 (such as 514) need root privileges on Linux, and Splunk should not run as root. Use a high port such as 5514 in labs. In production, send syslog to a dedicated syslog server (rsyslog, syslog-ng or SC4S), write it to files, and collect those files with a Universal Forwarder. You then keep the data even when Splunk restarts.

3.4 Introduction to the Universal Forwarder

The **Universal Forwarder (UF)** is Splunk's lightweight data-collection agent. It is a separate, small download that you install on the machines that produce the data.

Why use a forwarder?

- **Reliable:** buffers data if the indexer is down; supports indexer acknowledgement (`useACK`)
- **Efficient:** low CPU and memory; compresses data on the wire
- **Secure:** SSL/TLS between forwarder and indexer
- **Scalable:** load-balances across many indexers automatically
- **Centrally managed:** through the deployment server

Universal vs Heavy Forwarder

	UF	HF
Installer	Separate, small	Full Splunk Enterprise
Parses data	No*	Yes
Filter / mask / route	Limited	Yes
Splunk Web	No	Yes
Resource use	Very low	Moderate

*Except structured data (CSV/JSON) with `INDEXED_EXTRactions`.

Installing and configuring a UF on Linux

SHELL — ON THE NOVAMART WEB SERVER

```
# 1. Install (tgz shown; .rpm and .deb packages also exist)
sudo tar -xzf splunkforwarder-<version>-Linux-x86_64.tgz -C /opt
sudo /opt/splunkforwarder/bin/splunk start --accept-license # prompts for an admin user and password

# 2. Tell the forwarder where to send data (indexer receiving port 9997)
sudo /opt/splunkforwarder/bin/splunk add forward-server idx01.novamart.local:9997

# 3. Tell the forwarder what to collect
sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/httpd/access_log -index novamart -sourcetype access_combined

# 4. (Optional) Let the deployment server manage it from now on
sudo /opt/splunkforwarder/bin/splunk set deploy-poll ds01.novamart.local:8089

# 5. Start at boot, then check the connection
sudo /opt/splunkforwarder/bin/splunk enable boot-start
sudo /opt/splunkforwarder/bin/splunk list forward-server
```

These CLI commands simply write `.conf` files. The same configuration written by hand looks like this (the full files are in the dataset's `config` folder):

INPUTS.CONF

```
[monitor:///var/log/novamart/web/*_access.log]
index = novamart
sourcetype = access_combined
disabled = false
```

OUTPUTS.CONF

```
[tcpout]
defaultGroup = novamart_indexers

[tcpout:novamart_indexers]
server = idx01:9997, idx02:9997
```

WATCH OUT

The indexer must be **listening** before forwarders can connect: *Settings* → *Forwarding and receiving* → *Configure receiving* → *New Receiving Port* → *9997*. A forwarder pointing at a closed port is the most common reason why "data isn't arriving".

The deployment server in one minute

With hundreds of forwarders, you do not want to edit `inputs.conf` on each one. Instead:

1. Put configuration packages (*deployment apps*) in `$(SPLUNK_HOME)/etc/deployment-apps/` on the deployment server, e.g. `nm_web_inputs`.
2. Create a **server class** that selects clients by host name, IP or machine type, e.g. *all hosts matching* `web*`.
3. Map apps to server classes. Forwarders **phone home** on port 8089, download their apps, and restart if needed.

3.5 Data Validation and Basic Troubleshooting

Never assume new data arrived correctly. Run through this **validation checklist** every time you onboard a source:

#	Check	How
1	Did data arrive at all?	<code>index=novamart earliest=-15m</code> . If the data is historical, use <i>All time</i> .
2	Correct index, source type, host?	<code> tstats count where index=novamart by sourcetype, host</code>
3	Correct event count?	Compare with the source, e.g. <code>wc -l order_service.log = 3,042</code>
4	Correct event boundaries?	Look at <code>linecount</code> ; check for merged or split events
5	Correct timestamps?	Compare <code>_time</code> with the timestamp text in <code>_raw</code> ; look for events in the future; check <code>_indextime - _time</code>
6	Fields extracted?	Check the fields sidebar in Smart or Verbose mode

VALIDATION SEARCHES

```
| tstats count where index=novamart by sourcetype, host
```

```
| metadata type=sourcetypes index=novamart
```

```
index=novamart | eval lag_sec = _indextime - _time | stats min(lag_sec) max(lag_sec) avg(lag_sec) by sourcetype
```

```
index=novamart sourcetype=novamart:app | stats count by linecount
```

Common problems

Symptom	Likely cause	Fix
No data at all	Receiving port not enabled; firewall; wrong index name (the index does not exist); file permissions	Check <i>Forwarding and receiving</i> ; <code>splunk list forward-server</code> ; look for "index does not exist" messages; make the file readable by the Splunk user
Events appear at the wrong time (e.g. shifted by 5 h 30 m)	Timestamp has no time zone and <code>TZ</code> is not set	Set <code>TZ = Asia/Kolkata</code> for the source type, then re-index
"No results found", but data exists	The time range picker excludes the events (old or future-dated data)	Search <i>All time</i> ; fix the timestamp configuration

Symptom	Likely cause	Fix
Multi-line events split apart, or single lines merged	Wrong line-breaking settings	Set <code>LINE_BREAKER</code> / <code>SHOULD_LINEMERGE</code> ; test with the Add Data preview
Wrong source type (e.g. <code>too_small</code> , <code>log-2</code>)	Automatic source-type detection guessed	Assign the source type explicitly in <code>inputs.conf</code>
Duplicate events	The same file was uploaded twice, or <code>crcSalt</code> is misused	Delete and re-index; do not combine Upload with Monitor on the same file

TROUBLESHOOTING SEARCHES IN `_INTERNAL`

```
index=_internal sourcetype=splunkd log_level=WARN OR log_level=ERROR component=DateParserVerbose
```

```
index=_internal sourcetype=splunkd component=TailReader OR component=WatchedFile
```

```
index=_internal source=*metrics.log group=tcpin_connections | stats latest(_time) AS last_seen by hostname
```

CLI HELPERS

```
splunk btool inputs list --debug      # effective inputs and which file each setting comes from
splunk btool props list novamart:app --debug
splunk list monitor                  # what is being monitored
splunk list inputstatus              # read position of each monitored file
```

CASE STUDY 2

The 5 h 30 m mystery: onboarding the bastion host's secure log

Situation. A NovaMart engineer uploaded `secure.log` from `bastion01` to a Splunk Cloud trial whose indexers run on UTC. A security analyst then asked: "Were there any SSH logins between 02:00 and 03:00 on 17 September?" The search returned nothing, although the analyst could see the attack in the raw file.

Investigation. The syslog lines look like `Sep 17 02:34:51 bastion01 sshd[22275]: Accepted password for deploy from 203.0.113.77`. They have **no year and no time zone**. Splunk therefore read `02:34:51` as **UTC**. To the analyst, whose time zone is IST, the event appeared at **08:04:51**, 5 h 30 m later than it really happened.

```
index=novamart sourcetype=linux_secure "Accepted password" | eval raw_time=substr(_raw,1,15) | table
_time raw_time host
```

Fix. Declare the source's time zone for the source type, delete the wrongly indexed data, and re-upload:

```
[linux_secure]
TZ = Asia/Kolkata
```

In the Add Data wizard, the same setting is in *Set Source Type* → *Advanced* → *New setting*: `TZ` = `Asia/Kolkata`.

Lessons. (1) Always check that `_time` matches the timestamp text in `_raw`. (2) Timestamps without a time zone need `TZ`. (3) Timestamps are fixed at index time, so the fix requires re-indexing. That is why you onboard into a test index first.

TRY IT

Lab 2: upload the web, payment and authentication logs, monitor the application log, and (optionally) receive firewall syslog over UDP. **Lab 3:** validate everything with `tstats`, `metadata` and the checklist above.

KNOWLEDGE CHECK · MODULE 3

1. Name the three phases data goes through before it can be searched, and one thing that happens in each.
2. Which metadata field tells Splunk *how to parse* the data? Which one controls *retention and access*?
3. You uploaded a file with the wrong time zone. Can you fix the timestamps with a search-time setting? Explain.
4. Why is sending firewall syslog directly to a Splunk UDP input discouraged in production?
5. A forwarder is installed and configured, but no data arrives. List three things you would check.

Introduction to Search Processing Language (SPL)

Search structure · Terms, Booleans, wildcards, phrases · Time ranges · Field filters · The pipeline · fields, table, sort, rename, dedup, head · Transforming vs non-transforming

AFTER THIS MODULE YOU CAN

- read and write the basic structure of an SPL search
- combine terms with AND, OR and NOT, and use wildcards and exact phrases correctly
- restrict searches by time with the picker and with `earliest` / `latest`
- filter on field values with comparison operators and `IN`
- shape results with `fields`, `table`, `sort`, `rename`, `dedup` and `head`
- tell transforming commands from non-transforming commands

4.1 Search Structure

An SPL search is a **pipeline**. The first part retrieves events from the indexes; each `|` (pipe) passes the results to the next command, which changes them in some way.

```
index =novamart sourcetype =access_combined status =503 | stats count BY host | sort - count
```

search terms (implicit "search" command)
retrieve matching events from the index

command + arguments
count events per host

command
order the rows

35 events → 2 rows (web01: 20, web02: 15) → same 2 rows, sorted by count descending

Figure 4.1 — Anatomy of an SPL search

Component	Description	Example
Search terms	Keywords, phrases, field=value pairs and Booleans, placed before the first pipe	<code>index=novamart "timed out"</code>
Commands	Tell Splunk what to do with the results	<code>stats</code> , <code>table</code> , <code>sort</code>
Functions	Calculations used inside commands	<code>count</code> , <code>avg(latency_ms)</code> , <code>round()</code>
Arguments	Options that change a command's behaviour	<code>limit=5</code> , <code>showperc=false</code>
Clauses	Keywords that group or rename results	<code>BY host</code> , <code>AS revenue</code>

TIP · SEARCH EFFICIENCY, RULE 1

Filter as early as possible. Always put `index=` first, add `sourcetype=` and the most selective terms you know, and pick the narrowest time range that answers the question. Everything after the first pipe works on the events the search terms return, so fewer events in means a faster pipeline.

4.2 Terms, Boolean Operators, Wildcards and Exact Phrases

Search terms

A term is a word to look for in the raw event, e.g. `ERROR`, `checkout` or `gw-beta`. **Terms are case-insensitive:** `error`, `Error` and `ERROR` match the same events.

WATCH OUT · TERMS MATCH ANYWHERE

`index=novamart sourcetype=novamart:app ERROR` returns **153** events, although only 108 have the log level ERROR. Splunk splits text into terms at *minor breakers* such as `_ = - . :`, so `error_code=PAY_DECLINED` is indexed as the terms `error`, `code`, `pay` and `declined`, and the 45 declined-payment WARN events match too. When precision matters, filter on a **field** (e.g. `log_level=ERROR`) instead of a bare term.

Boolean operators

Operator	Meaning	NovaMart example
AND	Both terms must match. Implied between terms, so you rarely type it.	<code>ERROR payment-service</code> = <code>ERROR AND payment-service</code>
OR	Either term matches	<code>status=500 OR status=503</code>
NOT	Excludes events that match	<code>sourcetype=access_combined NOT status=200</code>
()	Groups expressions and overrides the default order	<code>(status=500 OR status=503) host=web01</code>

WATCH OUT · TWO RULES THAT TRIP EVERYONE UP

- Boolean operators must be UPPERCASE.** `error or warn` searches for three terms: `error`, `or` and `warn`.
- Order of evaluation:** parentheses first, then `NOT`, then `OR`, then `AND`. So `host=web01 status=500 OR status=503` means `host=web01 AND (status=500 OR status=503)`. Use parentheses whenever you are unsure.

Wildcards

The asterisk `*` matches zero or more characters, in terms and in field values.

Search	Matches
<code>status=40*</code>	401, 402, 403 and 404: 312 events
<code>uri_path=/cart*</code>	<code>/cart.do</code> (add, view and remove actions): 2,808 events
<code>sourcetype=novamart:*</code>	Both custom NovaMart source types
<code>fail*</code>	fail, failed, FAILED, failure...

TIP

Wildcards at the **end** of a term (`fail*`) are efficient. Wildcards at the **start** (`*fail`) or in the middle force Splunk to scan far more data. Avoid them on large indexes.

Exact phrases

Put a phrase in double quotes to match it exactly, in that word order:

```
index=novamart sourcetype=novamart:app "timed out"
```

This returns the **35** payment-gateway timeout events. Without quotes, `timed out` would match any event that contains both words anywhere, in any order.

Field names vs field values: case sensitivity

Element	Case-sensitive?
Search terms and field values (<code>status=failed</code>)	No
Field names (<code>Status=503</code> <code>≠</code> <code>status=503</code>)	Yes
Boolean operators (<code>AND</code> , <code>OR</code> , <code>NOT</code>)	Yes, must be uppercase
Command names (<code>STATS</code> = <code>stats</code>)	No, but lowercase is the convention

4.3 Time-Range Selection

Time is the most powerful filter in Splunk because data is stored in time-ordered buckets. A narrower time range means fewer buckets to open.

The time range picker

Option	Use for
Presets	Last 15 minutes, Last 24 hours, Today, Previous week, All time...
Relative	"Last 3 hours", optionally snapped to the start of the hour or day
Real-time	A sliding window that updates continuously (uses a lot of resources; use sparingly)
Date Range / Date & Time Range	Fixed windows, e.g. 18/09/2026 19:00:00 to 23:59:59
Advanced	Type time modifiers directly, e.g. <code>-7d@d</code> to <code>@d</code>

Inline time modifiers

You can put `earliest` and `latest` in the search itself. **They override the time range picker.**

Modifier	Meaning
<code>earliest=-24h</code>	From 24 hours ago until now
<code>earliest=-1d@d latest=@d</code>	All of yesterday (midnight to midnight). <code>@</code> "snaps" down to the unit.
<code>earliest=-7d@w1</code>	From Monday of last week (<code>w1</code> = Monday, <code>w0</code> = Sunday)
<code>earliest=@h</code>	From the start of the current hour
<code>earliest="09/18/2026:19:00:00"</code> <code>latest="09/18/2026:23:59:59"</code>	Absolute times, in <code>%m/%d/%Y:%H:%M:%S</code> format (month first)

EXAMPLE · ONLY THE FESTIVE SALE WINDOW

```
index=novamart sourcetype=access_combined earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"
```

Result: **7,218** events in five hours, about a third of the whole week's web traffic. Absolute times are interpreted in **your user time zone**.

WATCH OUT

Units: **s** seconds, **m** minutes, **h** hours, **d** days, **w** weeks, **mon** months, **y** years. **-1m** is one **minute**, not one month.

4.4 Field-Based Filtering

Filtering on fields is more precise than filtering on terms: `status=404` matches the HTTP status field only, whereas the bare term `404` would also match a byte count of 404 or a product ID such as NM-404.

Syntax	Example
<code>field=value</code>	<code>host=web01</code>
<code>field≠value</code>	<code>status≠200</code>
<code>> ≥ < ≤</code> (numeric)	<code>status ≥ 500</code> , <code>response_ms > 5000</code>
<code>field IN (v1, v2, ...)</code>	<code>payment_method IN (CARD, NETBANKING)</code>
<code>field="value with spaces"</code>	<code>category="Home & Kitchen"</code>
<code>field=*</code>	<code>error_code=*</code> (events where the field exists)

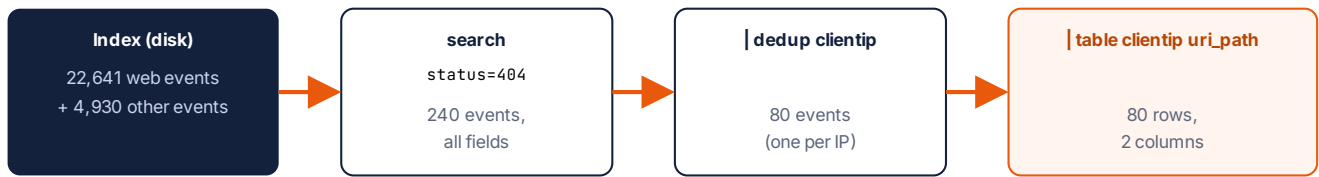
WATCH OUT · NOT FIELD=VALUE vs FIELD!=VALUE

`field≠value` returns events where the field **exists** and has a different value. `NOT field=value` also returns events where the field **does not exist at all**.

```
index=novamart sourcetype=novamart:app error_code≠PAY_DECLINED    `` 35 events: only events that HAVE error_code ``
index=novamart sourcetype=novamart:app NOT error_code=PAY_DECLINED `` 2,997 events: includes all events without error_code ``
```

4.5 The Search Pipeline and Commands

Think of each command as a stage in an assembly line. The output of one stage is a **table of results** that becomes the input of the next. Once a field has been removed or a row filtered out, later stages cannot see it.



Each stage passes a smaller or reshaped results table to the next

Figure 4.2 — Results shrinking as they move through the pipeline

4.6 Core Commands: fields, table, sort, rename, dedup, head

fields: keep or remove fields

SYNTAX

```
... | fields [+|-] field1, field2, ...
```

`fields +` (the default) keeps only the listed fields, and `fields -` removes them. Removing fields you don't need **early** in the pipeline makes searches faster, because Splunk then extracts and carries less data.

```
index=novamart sourcetype=access_combined status=503 | fields host, clientip, uri_path, status
index=novamart sourcetype=access_combined | fields - useragent, referer
```

TIP

Internal fields `_raw` and `_time` are always kept by `fields +` unless you remove them explicitly with `fields - _raw`.

table: build a table in a chosen column order

```
index=novamart sourcetype=access_combined status=503 | table _time, host, clientip, uri_path, status
```

_time	host	clientip	uri_path	status
2026-09-18 20:44:41	web02	157.207.95.172	/checkout	503
2026-09-18 20:43:37	web02	106.180.71.196	/checkout	503
2026-09-18 20:42:10	web02	223.219.23.45	/checkout	503
2026-09-18 20:41:12	web01	223.181.191.122	/checkout	503
... 35 rows in total				

`table` accepts wildcards (`table _time, error*`). The results appear on the **Statistics** tab.

WATCH OUT · FIELDS VS TABLE

`fields` only filters which fields travel down the pipeline, and it works on the indexers. `table` formats the output on the search head. Use `fields` early for performance and `table` at the end for presentation.

sort: order the results

SYNTAX

```
... | sort [limit] [+|-]field1, [+|-]field2 ...
```

- `sort - count` is descending; `sort count` or `sort + count` is ascending.
- Several fields: `sort host, -_time` sorts by host A-Z, then newest first within each host.
- By default, `sort` returns at most 10,000 results. Use `sort 0 field` for no limit.
- `num(field)`, `str(field)` and `ip(field)` force numeric, lexicographic or IP-address order.

```
index=novamart sourcetype=novamart:app action=checkout | table _time, order_id, amount, region | sort - amount
```

rename: give fields friendly names

```
index=novamart sourcetype=access_combined status=503
| table _time, host, clientip, uri_path
| rename clientip AS "Customer IP", uri_path AS "Page", host AS "Web Server"
```

- Use double quotes if the new name contains spaces.
- Wildcards work: `rename *_ms AS *_millis`.
- Rename at the **end** of the search. After a rename, later commands must use the *new* name.

dedup: remove duplicates

```
index=novamart sourcetype=access_combined status=404 | dedup clientip | table _time, clientip, uri_path
```

This returns **80** rows, one per distinct client IP that produced a 404. `dedup` keeps the **first** event it sees for each value. Because search results arrive newest first, that is the **most recent** event.

- Several fields: `dedup host, uri_path` keeps one event per unique combination.
- `dedup 3 clientip` keeps up to 3 events per IP.
- `keepempty=true` keeps events where the field is missing (they are dropped by default).
- `sortby -bytes` sorts the results after deduplication.

head (and tail): the first or last N results

```
index=novamart sourcetype=novamart:app ERROR | head 5
index=novamart sourcetype=novamart:app action=checkout | sort - amount | head 10
```

`head` returns the first 10 results by default. Placed early, it stops the search once N results are found, which makes it very fast for quick sampling. `tail N` returns the last N results (the oldest, in default order).

Command	One-line summary	NovaMart one-liner
<code>fields</code>	Keep (+) or drop (-) fields	<code> fields - _raw, useragent</code>
<code>table</code>	Tabular output with chosen columns	<code> table _time host status</code>
<code>sort</code>	Order the rows (default limit 10,000)	<code> sort 0 -amount</code>
<code>rename</code>	Change display names	<code> rename amount AS "Amount (INR)"</code>
<code>dedup</code>	One row per unique value (keeps the first seen)	<code> dedup customer_id</code>
<code>head</code>	First N results (default 10)	<code> head 20</code>

4.7 Transforming vs Non-Transforming Commands

Non-transforming (streaming / event-returning)

Work on events and return **events**, usually one event in, at most one event out. Results appear on the **Events** tab.

`search`, `where`, `fields`, `rename`, `eval`, `rex`, `dedup`, `sort`, `head`

Transforming

Turn events into a **statistical table** (new rows and columns). Results appear on the **Statistics** tab and can be charted on the **Visualization** tab.

`stats`, `chart`, `timechart`, `top`, `rare`, and `table` (which formats results as a table)

EXAMPLE · THE SAME DATA, TWO KINDS OF COMMAND

```
index=novamart sourcetype=access_combined | top limit=5 uri_path
```

uri_path	count	percent
/product.screen	7918	34.97
/category.screen	4381	19.35
/	3292	14.54
/cart.do	2808	12.40
/search	1281	5.66

`top` is transforming: 22,641 events become 5 rows with `count` and `percent` columns. Compare this with `... | sort uri_path | head 5`, which is non-transforming and returns 5 raw events.

```
index=novamart sourcetype=access_combined | stats count by status | sort - count
```

status	count
200	22256
404	240
402	45
500	38
503	35
403	14
401	13

TIP

Only transforming searches can be turned into charts, dashboard panels and most reports. When you want a visual, finish with `stats`, `chart`, `timechart`, `top` or `rare`. Day 2 covers these in depth.

Festive Sale night: triage in five searches

Situation. Friday 18 September, 20:15. Customer support reports that "card payments are failing" during the Festive Sale. You are on call. Here is how an experienced Splunk user narrows the problem down, using only this module's commands.

Step 1: Is there really a problem? Count server errors on the web tier.

```
index=novamart sourcetype=access_combined status>=500 earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"
| stats count by status
```

Result: `503` = 35 and `500` = 1. The 503 count is far above normal (there are no 503s at all on the other six days).

Step 2: Which pages and which servers?

```
index=novamart sourcetype=access_combined status=503 | stats count by uri_path, host
```

Result: every 503 is on `/checkout`, spread across **both** web01 (20) and web02 (15). This is not a single bad server, so the cause is more likely a shared dependency.

Step 3: When exactly?

```
index=novamart sourcetype=access_combined status=503 | sort _time | head 1
index=novamart sourcetype=access_combined status=503 | sort - _time | head 1
```

Result: first at `20:07:38`, last at `20:44:41`.

Step 4: What does the application say?

```
index=novamart sourcetype=novamart:app ERROR earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"
| table _time, _raw | head 5
```

Result: repeated `Payment gateway gw-beta timed out after 30000 ms` messages from `payment-service`.

Step 5: Who was affected?

```
index=novamart sourcetype=novamart:app status=FAILED
| dedup customer_id
| table _time, customer_id, order_id, amount, payment_method
| sort - amount
| rename customer_id AS Customer, order_id AS Order, amount AS "Amount (INR)", payment_method AS Method
```

Result: 35 customers, all paying by **CARD** or **NETBANKING**, the two methods routed to `gw-beta`.

Conclusion: the payment gateway `gw-beta` was timing out, and the web tier returned 503 on checkout. In Module 5 you will extract the gateway name as a field and put a rupee figure on the failed orders.

TRY IT

Lab 4 practises terms, Booleans, wildcards, phrases, time ranges and field filters. **Lab 5** practises `fields`, `table`, `sort`, `rename`, `dedup`, `head` and your first transforming commands.

KNOWLEDGE CHECK · MODULE 4

1. How does Splunk interpret `index=novamart error OR warn host=web01`? Rewrite it so that it means "(error or warn) on web01" without any ambiguity.
2. What is the difference between `NOT status=200` and `status≠200`? When would the results differ?
3. You sort 50,000 events by `amount` but get only 10,000 rows. Why, and how do you fix it?
4. Which event does `dedup clientip` keep for each IP address?
5. Is `top` transforming or non-transforming? How can you tell from the Search app?
6. Write a search for all 4xx web events on web02 between 18 Sep 19:00 and 19 Sep 00:00.

Fields and Knowledge Objects

Default, interesting and discovered fields · Field Extractor · rex · eval, case, if · Tags, event types and lookups

AFTER THIS MODULE YOU CAN

- tell default, interesting and discovered fields apart
- create a field extraction with the Field Extractor, and write one with `rex`
- create calculated values with `eval`, `if` and `case`
- explain what tags, event types and lookups are, and create a simple one of each

5.1 Default, Interesting and Discovered Fields

A field is a searchable **name=value** pair. Splunk gets fields in three ways:

Category	What it is	Examples
Default fields	Added by Splunk to every event at index time, whatever the data	<code>host</code> , <code>source</code> , <code>sourcetype</code> , <code>index</code> , <code>_time</code> , <code>_raw</code> , <code>linecount</code> , <code>punct</code> , <code>splunk_server</code> , <code>date_hour</code> , <code>date_wday</code> ...
Discovered (extracted) fields	Extracted at search time from the raw text by automatic key=value detection, source-type definitions (e.g. <code>access_combined</code>), JSON parsing, or your own extractions	<code>clientip</code> , <code>status</code> , <code>uri_path</code> (access_combined); <code>amount</code> , <code>region</code> (key=value); <code>gateway</code> , <code>latency_ms</code> (JSON)
Interesting fields	A <i>display</i> category in the fields sidebar: discovered fields that appear in at least 20% of the events in your results	For web events: <code>bytes</code> , <code>clientip</code> , <code>method</code> , <code>status</code> ...

The sidebar also has **Selected fields**. These are shown under every event in the list view; by default they are `host`, `source` and `sourcetype`. Click any field and choose *Selected: Yes* to add it. **All Fields** opens the complete list, including fields found in fewer than 20% of events.

In the field pop-up

- **a** = string field, **#** = numeric field
- Top 10 values with count and percentage
- Quick reports: *Top values*, *Top values by time*, *Rare values*, *Events with this field*
- For numeric fields: *Average over time*, *Maximum/Minimum value over time*

Why search-time extraction matters

- Fields cost nothing extra at index time and use no extra licence
- You can add new fields to *old* data at any time
- If an extraction is wrong, just fix it; no re-indexing needed

EXAMPLE · FIELDS YOU GET FOR FREE IN NOVAMART DATA

Source type	Mechanism	Fields available with no configuration
<code>access_combined</code>	Pretrained source type	<code>clientip</code> , <code>method</code> , <code>uri</code> , <code>uri_path</code> , <code>status</code> , <code>bytes</code> , <code>referer</code> , <code>useragent</code> , plus <code>action</code> , <code>product_id</code> , <code>category_id</code> , <code>order_id</code> , <code>q</code> from the URL query string (key=value)
<code>novamart:app</code>	<code>KV_MODE=auto</code>	<code>txn_id</code> , <code>customer_id</code> , <code>action</code> , <code>order_id</code> , <code>product_id</code> , <code>qty</code> , <code>amount</code> , <code>payment_method</code> , <code>status</code> , <code>error_code</code> , <code>response_ms</code> , <code>region</code>
<code>novamart:payment</code>	JSON	<code>gateway</code> , <code>order_id</code> , <code>amount</code> , <code>payment_method</code> , <code>result</code> , <code>latency_ms</code> , <code>card_network</code> , <code>decline_reason</code> , <code>error.code</code> , <code>error.retries</code>
<code>linux_secure</code>	key=value (sudo lines only)	<code>TTY</code> , <code>PWD</code> , <code>USER</code> , <code>COMMAND</code> . The IP addresses and user names in <code>sshd</code> messages are not extracted, so you will extract them yourself in 5.3.

5.2 Field Extraction with the Field Extractor (IFX)

When a value you need is buried in free text, you can teach Splunk to extract it. The **Field Extractor** is a point-and-click wizard that writes the regular expression for you. Open it with:

- *Event Actions* → *Extract Fields* on an expanded event, or
- **+ Extract New Fields** at the bottom of the fields sidebar, or
- *Settings* → *Fields* → *Field extractions* → *Open Field Extractor*.

- 1 Select sample.** Choose the source type and a representative event, e.g. a `payment-service` timeout line.
- 2 Select method.** *Regular Expression* (for unstructured text) or *Delimiters* (for CSV-like data separated by commas, pipes, tabs or spaces).
- 3 Select fields.** Highlight the value in the sample and name the field, e.g. highlight `gw-beta` and call it `gateway`; highlight `30000` and call it `timeout_ms`. Splunk generates a regex and previews matches across many events.
- 4 Validate.** Check the preview. Mark wrong matches with **X** to add counter-examples, or click *Show regular expression* and edit it by hand.
- 5 Save.** Give the extraction a name and set permissions: *Owner* (private), *App* or *All apps*. The fields now exist in every search on that source type, including searches of data indexed before you created the extraction.

SAMPLE EVENT

```
2026-09-18T20:08:08.464+05:30 ERROR [payment-service] txn_id=TXN-101536 Payment gateway gw-beta timed out after 30000 ms for order ORD-550497 attempt 3 of 3
```

WHAT IFX SAVES BEHIND THE SCENES (PROPS.CONF ON THE SEARCH HEAD)

```
[novamart:app]
EXTRACT-gw_timeout = gateway (?<gateway>\S+) timed out after (?<timeout_ms>\d+) ms for order (?<order_id>ORD-\d+)
```

TIP

The Field Extractor is a great way to *learn* regex: build an extraction with it, then click *Show regular expression* to see what it wrote. Make sure the sample you pick is representative. An extraction built from an ERROR line will not match INFO lines that have a different layout.

5.3 Field Extraction with rex

The `rex` command extracts fields **inside a search** using a regular expression with **named capture groups**. It suits ad-hoc investigations and testing a pattern before you save it as a permanent extraction.

SYNTAX

```
... | rex [field=<field>] "<regex with (?<name>...) groups>" [max_match=<n>]
... | rex [field=<field>] mode=sed "s/<regex>/<replacement>/g"
```

- `field=` sets the field to read from; the default is `_raw`.
- `(?<name>pattern)` creates a field called *name* from whatever *pattern* matches.
- `max_match` is the number of matches to capture (default 1; `0` = unlimited, which makes a multivalue field).
- `mode=sed` replaces text, e.g. to mask data in search results.

Regular-expression quick reference

Token	Matches	Token	Matches
<code>\d</code>	a digit 0–9	<code>+</code>	one or more of the previous item
<code>\w</code>	a letter, digit or underscore	<code>*</code>	zero or more
<code>\s</code> / <code>\S</code>	whitespace / non-whitespace	<code>?</code>	zero or one (optional)
<code>.</code>	any character	<code>{3}</code>	exactly 3 times
<code>[^\\]</code>	any character except <code>]</code>	<code>^</code> / <code>\$</code>	start / end of the text
<code>\.</code> <code>\[</code>	a literal dot or bracket (escaped)	<code>(a b)</code>	a or b

EXAMPLE 1 · LOG LEVEL AND SERVICE NAME FROM THE APPLICATION LOG

```
index=novamart sourcetype=novamart:app
| rex "^\\S+\\s+(?<log_level>[A-Z]+)\\s+\\[(?<service>[^\]]+\\)\\]"
| stats count by service, log_level
```

service	log_level	count
auth-service	INFO	435
cart-service	INFO	1404
inventory-service	WARN	60
order-service	ERROR	35
order-service	INFO	807
order-service	WARN	50
payment-service	ERROR	35
payment-service	INFO	1
payment-service	WARN	9
platform	INFO	168
search-service	ERROR	38

How to read the regex: `^\\S+` skips the timestamp, `\\s+` skips the spaces, `(?<log_level>[A-Z]+)` captures the capital-letter word, and `\\[(?<service>[^\]]+\\)\\]` captures everything between the square brackets.

EXAMPLE 2 · WHICH GATEWAY TIMED OUT?

```
index=novamart sourcetype=novamart:app "timed out"
| rex "gateway (?<gateway>\\S+) timed out after (?<timeout_ms>\\d+) ms"
| stats count by gateway, timeout_ms
```

Result: a single row, `gw-beta` | `30000` | `35`. Every timeout was on gw-beta, after a 30-second timeout.

EXAMPLE 3 · USER AND SOURCE IP FROM SSH MESSAGES

```
index=novamart sourcetype=linux_secure "Failed password"
| rex "Failed password for (invalid user )?(?<user>\\S+) from (?<src_ip>\\d{1,3}\\.\\d{1,3}\\.{3})"
| stats count by src_ip, user
| sort - count
| head 5
```

`(invalid user )?` makes that phrase optional, so a single pattern handles both *"Failed password for root from..."* and *"Failed password for invalid user admin from..."*.

EXAMPLE 4 · MASKING SESSION IDS WITH MODE=SED

```
index=novamart sourcetype=access_combined | rex field=_raw mode=sed "s/JSESSIONID=\w+/JSESSIONID=XXX/g" | table _raw | head 3
```

This changes the *search results only*. To mask data before it is stored, use `SEDCMD` or `TRANSFORMS` in `props.conf` at parse time.

WATCH OUT · REX VS PERMANENT EXTRACTIONS

`rex` exists only for the search it is in. If you use the same `rex` in many searches, save it as a field extraction (IFX, or *Settings* → *Fields* → *Field extractions* → *New*) so that everyone gets the field automatically and the logic lives in one place.

5.4 Calculated Values with eval, if and case

`eval` calculates an expression and puts the result in a field. If the field exists, it is overwritten; if not, it is created. It works like a spreadsheet formula applied to every event.

SYNTAX

```
... | eval <field> = <expression> [, <field2> = <expression2> ...]
```

Category	Operators / functions	Example
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>eval amount_k = amount / 1000</code>
String concatenation	<code>.</code> (dot)	<code>eval label = host . ":" . uri_path</code>
Comparison / Boolean	<code>=</code> <code>≠</code> <code><</code> <code>></code> <code>AND</code> <code>OR</code> <code>NOT</code>	<code>if(status ≥ 500 AND host="web01", ...)</code>
Mathematical	<code>round(x,n)</code> , <code>abs</code> , <code>ceiling</code> , <code>floor</code>	<code>eval sec = round(response_ms/1000, 2)</code>
Text	<code>lower</code> , <code>upper</code> , <code>len</code> , <code>substr</code> , <code>replace</code> , <code>split</code>	<code>eval svc = upper(service)</code>
Conditional	<code>if(cond, then, else)</code> , <code>case(c1,v1, c2,v2, ...)</code> , <code>coalesce</code>	see below
Date / time	<code>now()</code> , <code>strftime</code> , <code>strptime</code> , <code>relative_time</code>	<code>eval day = strftime(_time, "%a %d-%b")</code>
Type / null	<code>tonumber</code> , <code>tostring</code> , <code>isnull</code> , <code>isnotnull</code> , <code>typeof</code>	<code>eval inr = tostring(amount, "commas")</code>

if(): two outcomes

```
index=novamart sourcetype=access_combined
| eval outcome = if(status ≥ 400, "error", "ok")
| stats count by outcome
```

Result: `ok` = 22,256 and `error` = 385.

case(): many outcomes

`case()` checks each condition in turn and returns the value paired with the **first** true condition. Finish with `true(), "default"` to catch everything else. Without it, events that match no condition get a null value.

```

index=novamart sourcetype=access_combined
| eval status_class = case(status < 300, "2xx Success",
                           status < 400, "3xx Redirect",
                           status < 500, "4xx Client error",
                           true(),      "5xx Server error")
| stats count by status_class

```

status_class	count
2xx Success	22256
4xx Client error	312
5xx Server error	73

EXAMPLE · BUSINESS VIEW: ORDER VALUE TIERS AND RESPONSE TIME IN SECONDS

```

index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| eval order_tier = case(amount ≥ 10000, "High (10k+)",
                        amount ≥ 2000, "Medium (2k-10k)",
                        true(),      "Low (<2k)",
                        response_sec = round(response_ms / 1000, 2),
                        slow = if(response_ms > 5000, "yes", "no")
| stats count, sum(amount) AS revenue by order_tier

```

Order counts: Low = 343, Medium = 334, High = 135. The 135 high-value orders (mostly smartphones and TVs bought during the sale) bring in the largest share of revenue.

WATCH OUT · QUOTES IN EVAL

In `eval`, **double quotes** mean a string literal and **single quotes** mean a field name. So `eval x = "status"` sets `x` to the word `status`, while `eval x = 'status'` copies the value of the `status` field. You need single quotes for field names that contain spaces or special characters, e.g. `'error.code'`. Field comparisons inside `eval` and `if` use `=`, although `=` also works.

Calculated fields: eval as a knowledge object

If an `eval` expression is useful everywhere, save it as a **calculated field** (*Settings* → *Fields* → *Calculated fields* → *New*). Splunk then evaluates it automatically on every search of that source type:

```

[novamart:app]
EVAL-response_sec = round(response_ms/1000, 2)

```

5.5 Introduction to Tags, Event Types and Lookups

Knowledge objects are saved definitions that add meaning to your data. They are created by users, stored on the search head, and shared by setting permissions (Private, App or All apps). Today we introduce three of them.

Event types: save a search as a category

An **event type** is a named search that classifies events. Any event that matches the search gets an extra field, `eventtype=<name>`.

DEFINITION: SETTINGS → EVENT TYPES → NEW (OR EVENT ACTIONS → BUILD EVENT TYPE)

Name: nm_checkout_failed

Search: index=novamart sourcetype=novamart:app action=checkout status=FAILED

USE

eventtype=nm_checkout_failed | stats count by payment_method

Rules: an event-type search cannot contain pipes or subsearches, only search terms. You can give it a colour, and a priority for when several event types match the same event.

Tags: friendly labels for field values

A **tag** attaches a label to a specific **field=value** pair. Tags let you search by meaning instead of by exact values, and they let different data sources share a common vocabulary.

Field=value	Tag	Why
host=bastion01	critical_asset	Security can search tag=critical_asset without remembering host names
host=web01, host=web02	web_tier	One tag covers all web servers, including future ones
eventtype=nm_checkout_failed	failure	Tag event types too, e.g. tag=failure across all sources

tag=web_tier status ≥ 500 | stats count by host

tag::host=critical_asset "Accepted password"

Create a tag from an expanded event: click the *Actions* arrow next to a field value → *Edit Tags*. Or use *Settings* → *Tags*. Tag names are case-sensitive.

Lookups: enrich events with external data

A **lookup** adds fields to your events from an external table, for example a CSV file. NovaMart's application log contains only product_id=NM-1001, but the business wants product names and categories. The product catalogue is in novamart_products.csv:

NOVAMART_PRODUCTS.CSV (FIRST ROWS)

```
product_id,product_name,category,unit_price,supplier
NM-1001,Aurora 5G Smartphone 128GB,Electronics,18999,Zenlite Devices
NM-1002,PulseBeat Wireless Earbuds,Electronics,2499,Zenlite Devices
NM-2001,Men's Slim Fit Denim Jeans,Fashion,1499,Loom & Thread
```

- 1 **Upload the table file:** *Settings* → *Lookups* → *Lookup table files* → *New*. Choose the CSV and a destination file name.
- 2 **Create a lookup definition** (recommended): *Settings* → *Lookups* → *Lookup definitions* → *New*. Name novamart_products, type *File-based*, file novamart_products.csv.
- 3 **Share it:** set permissions on the file and the definition (e.g. App: Search & Reporting, read for Everyone).
- 4 **Optional: make it automatic:** *Settings* → *Lookups* → *Automatic lookups*, so every novamart:app event gets the product fields without the lookup command.

CHECK THE TABLE, THEN USE IT

```
| inputlookup novamart_products
```

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
```

```
| lookup novamart_products product_id OUTPUT product_name, category
```

```
| stats sum(amount) AS revenue by category
```

```
| sort - revenue
```

category	revenue
Electronics	4227221
Home & Kitchen	342070
Fashion	297862
Sports	218723
Beauty	67980
Books	38642

- **OUTPUT** overwrites fields that already exist in the event; **OUTPUTNEW** only fills in fields that are missing.
- If the event's field name differs from the lookup column, map it with **AS**: `lookup threat_intel_ips ip AS src_ip OUTPUT threat_category` matches the lookup column `ip` against the event field `src_ip`.
- Other lookup types include KV Store lookups, external (scripted) lookups and geospatial lookups.

CASE STUDY 4

2 AM at the bastion: from raw text to a security finding

Situation. On Monday morning, NovaMart's security lead asks you to review last week's SSH activity on `bastion01`, the only way into the production network. The `linux_secure` data has none of the fields you need: user and source IP are buried in free text.

1. Extract the fields.

```
index=novamart sourcetype=linux_secure ("Failed password" OR "Accepted")
| rex "(?<auth_result>Failed|Accepted) (password|publickey) for (invalid user )?(?<user>\S+) from (?<src_ip>\S+)"
| stats count by src_ip, auth_result
| sort - count
```

One IP stands out: `203.0.113.77` with **414 failed** passwords and, worryingly, **1 accepted** login.

2. Enrich with threat intelligence (lookup).

```
... | lookup threat_intel_ips ip AS src_ip OUTPUT threat_category, confidence
```

Result: `203.0.113.77` → `ssh_bruteforce`, confidence `high`.

3. Classify with eval.

```

index=novamart sourcetype=linux_secure "203.0.113.77"
| rex "for (invalid user )?(?<user>\S+) from"
| eval attempt_type = case(like(_raw, "%Invalid user%"), "invalid-user probe",
                           like(_raw, "%Failed password%"), "password guess",
                           like(_raw, "%Accepted%"), "SUCCESSFUL LOGIN",
                           true(), "other")
| stats count, min(_time) AS first_seen, max(_time) AS last_seen by attempt_type
| eval first_seen=strftime(first_seen, "%d-%b %H:%M:%S"), last_seen=strftime(last_seen, "%d-%b %H:%M:%S")

```

4. What happened after the login?

```

index=novamart sourcetype=linux_secure sudo TTY=pts/3 earliest="09/17/2026:02:30:00" latest="09/17/2026:03:00:00"
| rex "sudo:\s+(?<sudo_user>\S+) :.*COMMAND=(?<full_command>.+)$"
| table _time, sudo_user, USER, full_command

```

The attacker logged in as `deploy` at **02:34:51** on 17 September, after about 23 minutes of guessing across 12 user names. Within two minutes they ran `/bin/cat /etc/shadow` as root and created a new account with `useradd -m -s /bin/bash support2`.

WATCH OUT

The automatically extracted `COMMAND` field holds only `/usr/sbin/useradd`, because automatic key=value extraction stops at the first space. `USER` is the *target* user (root), not the person who ran sudo. That is why the search extracts `sudo_user` and `full_command` with `rex`.

Outcome and lessons. Security disabled `deploy` password logins, removed `support2` and blocked the IP. The evidence had been sitting in the logs for four days: `rex` gave it structure, a *lookup* gave it context, and `eval` made the story readable. An **event type** `nm_ssh_bruteforce` plus an alert (Day 2) would have caught it within minutes.

TRY IT

Lab 6: build the payment-gateway extraction with the Field Extractor, write `rex` extractions for the application and SSH logs, create calculated fields with `eval`, `if` and `case`, and create your first event type, tag and lookup. **Lab 7** puts everything together in a full incident investigation.

KNOWLEDGE CHECK · MODULE 5

1. What is the difference between an *interesting* field and a *discovered* field?
2. Write a `rex` that extracts `order_id` (values like `ORD-550497`) from any event.
3. What does `case()` return for an event that matches none of its conditions if you leave out `true()`?
4. In `eval`, what is the difference between `"amount"` and `'amount'`?
5. Can an event-type definition contain `| stats`? Why or why not?
6. When would you use `OUTPUTNEW` instead of `OUTPUT` in a lookup?

Day 1 Summary

5

MODULES

7

HANDS-ON LABS

27,571

EVENTS ONBOARDED

2

INCIDENTS UNCOVERED

Module	Key takeaways
1 · Introduction	Splunk indexes machine data as it arrives and structures it at search time (schema-on-read). Forwarders collect, indexers store and search, search heads coordinate and present, the deployment server manages configuration. Licensing is by daily ingest or by compute; overages never stop indexing.
2 · User interface	Search & Reporting is home base. Choose Fast, Smart or Verbose mode on purpose. Use the timeline and fields sidebar to explore. The Job Inspector explains slow searches (compare <code>scanCount</code> with <code>eventCount</code>). <code>_time</code> is stored in UTC and shown in your time zone.
3 · Onboarding	Data goes through input → parsing → indexing → search. Set index, source type and host deliberately. Upload once, monitor continuously, receive network data via a syslog server. Validate every new source; timestamps and line breaks are fixed at index time.
4 · SPL basics	Filter early (<code>index</code> , <code>sourcetype</code> , time). Boolean operators are uppercase, and NOT → OR → AND is the evaluation order. <code>≠</code> is not the same as <code>NOT</code> . Shape results with <code>fields</code> , <code>table</code> , <code>sort</code> , <code>rename</code> , <code>dedup</code> and <code>head</code> . Transforming commands produce statistics tables.
5 · Fields & KOs	Most fields are extracted at search time. Build extractions with the Field Extractor or <code>rex</code> ; calculate with <code>eval</code> , <code>if</code> and <code>case</code> . Event types classify, tags label, lookups enrich.

Coming up on Day 2

Statistical commands in depth (`stats` , `chart` , `timechart`), reports and visualisations, dashboards, and alerts. You will turn today's investigations into a *NovaMart Operations* dashboard and a brute-force alert.

SPL Quick Reference

SEARCH BASICS

<code>index=novamart</code> <code>sourcetype=access</code> <code>_combined</code>	Always start with index and source type
<code>"timed out"</code>	Exact phrase
<code>fail*</code>	Wildcard (put it at the end)
<code>a OR b</code> , <code>NOT c</code> , <code>()</code>	Uppercase Booleans; evaluated in the order <code>()</code> → <code>NOT</code> → <code>OR</code> → <code>AND</code>
<code>status ≥ 500</code>	Numeric comparison
<code>method IN (GET, POST)</code>	List of values
<code>field=*</code>	Field exists
<code>earliest=-24h@h</code> <code>latest=now</code>	Inline time range

SHAPING RESULTS

<code> fields - _raw</code>	Drop fields (early, for speed)
<code> table a b c</code>	Tabular output
<code> sort 0 -amount</code>	Sort descending, no limit
<code> rename a AS "Nice Name"</code>	Display names
<code> dedup clientip</code>	One (most recent) per value
<code> head 10</code> / <code> tail 10</code>	First / last N
<code> top limit=5 uri_path</code>	Most common values
<code> rare useragent</code>	Least common values
<code> stats count by host</code>	Count per group

FIELDS AND CALCULATIONS

<code> rex "id=(?<id>\d+)"</code>	Extract with regex
<code> rex field=f mode=sed</code> <code>"s/x/y/g"</code>	Replace / mask
<code> eval s = round(ms/1000, 2)</code>	Calculate
<code> eval r = if(c, "y", "n")</code>	Two outcomes
<code> eval r = case(c1,"a", c2,"b", true(),"z")</code>	Many outcomes
<code> eval t = strftime(_time, "%F %T")</code>	Format time
<code> lookup def key OUTPUT f1 f2</code>	Enrich
<code> inputlookup def</code>	View a lookup table
<code>eventtype=name</code> , <code>tag=name</code>	Use knowledge objects

VALIDATION AND TROUBLESHOOTING

<code> tstats count where index=x</code> <code>by sourcetype</code>	Fast metadata counts
<code> metadata type=hosts</code> <code>index=x</code>	Hosts and first/last seen
<code> eval lag=_indextime-_time</code>	Indexing delay
<code>index=_internal</code> <code>log_level=ERROR</code>	Splunk's own errors
<code>splunk btool props list st --</code> <code>debug</code>	Effective configuration

TIME MODIFIER UNITS

`s` sec · `m` min · `h` hour · `d` day · `w` week · `mon` month · `y` year · `@d` snap to midnight · `@w1` snap to Monday

Glossary

Term	Definition
Bucket	A directory of index data covering a time span; ages hot → warm → cold → frozen.
Deployment server	Splunk instance that distributes apps and configurations to forwarders and other clients.
Event	A single record of data, with a timestamp, in an index.
Event type	A saved search, without pipes, that labels matching events with <code>eventtype=<name></code> .

Term	Definition
Field	A searchable name=value pair extracted from, or added to, an event.
Fishbucket	Internal database recording how far Splunk has read each monitored file.
Forwarder	Instance that sends data to indexers: Universal (lightweight) or Heavy (full, can parse).
HEC	HTTP Event Collector: token-based HTTP(S) data input, port 8088 by default.
Index	A repository of events on disk; controls retention and access.
Indexer	Instance that parses and stores data and runs searches against it.
Knowledge object	User-defined object that adds meaning to data: saved searches, field extractions, event types, tags, lookups, calculated fields...
Lookup	Table (CSV, KV Store or script) used to add fields to events.
Schema-on-read	Applying structure (fields) at search time rather than when data is stored.
Search head	Instance that provides the UI, coordinates searches and merges the results.
Source / Source type / Host	Default fields: where the data came from / its format / the machine that produced it.
SPL	Search Processing Language: Splunk's pipeline-based query language.
Tag	A label attached to a field=value pair.
Transforming command	A command that turns events into a statistical results table (e.g. <code>stats</code> , <code>top</code> , <code>chart</code>).
tsidx	Time-series index files that map search terms to event locations in raw data.

Appendix A: Knowledge Check Answers

Module 1

1. Structure (fields) is applied when you search, not when data is stored. Machine data comes in endless, changing formats. Schema-on-read lets you index it immediately and ask new questions later without redesigning anything.
2. The **Universal Forwarder** collects the logs. The **deployment server** manages the 200 installations centrally, through server classes and deployment apps.
3. A **Heavy Forwarder**, because masking before data leaves the site requires parsing (`SEDCMD` or `TRANSFORMS`), and the UF does not parse unstructured data. (Masking can also be done on the indexers, but then the data has already left the data centre.)
4. **False**. Indexing continues. Depending on licence type and size, searching may be restricted after repeated violations.
5. Any two of: in Cloud there is no CLI or file-system access to the stack; apps need vetting; Splunk handles upgrades, scaling and infrastructure; configuration is done through Splunk Web or the ACS API rather than `.conf` files.

Module 2

1. **Verbose**, because it discovers all fields and keeps the events, which helps when you don't yet know what the data contains. (Smart mode is also fine for plain event searches.)
2. Ad-hoc jobs expire 10 minutes after they finish. Extend the job's lifetime, share it (7 days) or save it as a report.
3. The search read 2 million events to find 35, which is very inefficient. Add `index` and `sourcetype`, more specific terms and field filters, and a narrower time range.
4. An app has a user interface (dashboards, navigation) for a use case. An add-on usually has no UI, and provides inputs, source types and field extractions for one technology.
5. No. `_time` is stored in UTC and each user sees it in the time zone set in their own Preferences. London is UTC+1 in summer (BST), so the difference is 4 h 30 m; it is 5 h 30 m in winter.

Module 3

1. Input (read the data, attach metadata), parsing (break into events, extract timestamps, transforms), indexing (write compressed raw data and tsidx files to buckets).
2. `sourcetype` controls parsing. `index` controls retention and access.
3. No. `_time` is set at index time. Fix the configuration (`TZ`), delete the data and re-index it.
4. UDP can lose packets, data is lost while Splunk restarts, and ports below 1024 need root. A syslog server (or SC4S) writing to files that a UF reads is more reliable.
5. Any three of: is receiving enabled on the indexer (9997)? Is the forwarder configured with the right server (`splunk list forward-server`)? Is there a firewall in between? Does the target index exist? Can the Splunk user read the file? What do `splunkd.log` and `_internal` say?

Module 4

1. As `index=novamart AND (error OR warn) AND host=web01`, because OR is evaluated before AND. To make it unambiguous: `index=novamart (error OR warn) host=web01`.
2. `status≠200` requires the status field to exist. `NOT status=200` also returns events that have no status field. The results differ whenever some events lack the field.
3. `sort` returns at most 10,000 results by default. Use `sort 0 -amount`.
4. The first one it encounters, which in default reverse-time order is the **most recent** event for that IP.

5. Transforming. Its results appear on the Statistics tab (with count and percent columns) and can be visualised.
6. `index=novamart sourcetype=access_combined host=web02 status ≥ 400 status < 500`
`earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"` (23 events: 13 × 402, 10 × 404).

Module 5

1. A discovered field is any field extracted at search time. "Interesting" is only a sidebar category: discovered fields that appear in at least 20% of the returned events.
2. `| rex "(?<order_id>ORD-\d+)"`
3. A null value: the field is not created for that event.
4. `"amount"` is the literal string *amount*; `'amount'` is the value of the field named amount.
5. No. An event type is a filter that is checked against individual events, so it can contain only search terms (no pipes or subsearches).
6. When events may already have the field and you want to keep the event's own value, and fill it from the lookup only when it is missing.